

Aditya ML Lab 4

NAME-Aditya Bhende

Sec-C-C2

ROLL NO.-28

Code

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

from sklearn.inspection import permutation_importance
```

Code

```
df = pd.read_csv("WineQT.csv")
```

Code

```
df.head()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	7.4	0.70	0.00	1.9	0.076	
1	7.8	0.88	0.00	2.6	0.098	
2	7.8	0.76	0.04	2.3	0.092	
3	11.2	0.28	0.56	1.9	0.075	
4	7.4	0.70	0.00	1.9	0.076	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	11.0	34.0	0.9978	3.51	0.56	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9970	3.26	0.65	
3	17.0	60.0	0.9980	3.16	0.58	
4	11.0	34.0	0.9978	3.51	0.56	

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4

Code

```
df.tail()
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
1138	6.3	0.510	0.13	2.3	0.076	
1139	6.8	0.620	0.08	1.9	0.068	
1140	6.2	0.600	0.08	2.0	0.090	
1141	5.9	0.550	0.10	2.2	0.062	
1142	5.9	0.645	0.12	2.0	0.075	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
1138	29.0	40.0	0.99574	3.42	0.75	
1139	28.0	38.0	0.99651	3.42	0.82	
1140	32.0	44.0	0.99490	3.45	0.58	
1141	39.0	51.0	0.99512	3.52	0.76	
1142	32.0	44.0	0.99547	3.57	0.71	

	alcohol	quality	Id
--	---------	---------	----

1138	11.0	6	1592
1139	9.5	6	1593
1140	10.5	5	1594
1141	11.2	6	1595
1142	10.2	5	1597

Code

```
df.shape

(1143, 13)
```

Code

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   fixed acidity        1143 non-null   float64
1   volatile acidity     1143 non-null   float64
2   citric acid          1143 non-null   float64
3   residual sugar       1143 non-null   float64
4   chlorides            1143 non-null   float64
5   free sulfur dioxide  1143 non-null   float64
6   total sulfur dioxide 1143 non-null   float64
7   density              1143 non-null   float64
8   pH                  1143 non-null   float64
9   sulphates            1143 non-null   float64
10  alcohol              1143 non-null   float64
11  quality              1143 non-null   int64   
12  Id                   1143 non-null   int64   
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

Code

```
df.describe()

      fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1143.000000      1143.000000    1143.000000      1143.000000
mean         8.311111         0.531339      0.268364         2.532152
std         1.747595         0.179633      0.196686         1.355917
min          4.600000         0.120000      0.000000         0.900000
25%          7.100000         0.392500      0.090000         1.900000
50%          7.900000         0.520000      0.250000         2.200000
75%          9.100000         0.640000      0.420000         2.600000
max         15.900000         1.580000      1.000000        15.500000

      chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count    1143.000000      1143.000000      1143.000000    1143.000000
mean         0.086933         15.615486         45.914698      0.996730
std          0.047267         10.250486         32.782130      0.001925
min          0.012000          1.000000          6.000000      0.990070
25%          0.070000          7.000000         21.000000      0.995570
50%          0.079000         13.000000         37.000000      0.996680
75%          0.090000         21.000000         61.000000      0.997845
max          0.611000         68.000000        289.000000      1.003690

      pH  sulphates  alcohol  quality      Id
count    1143.000000    1143.000000    1143.000000    1143.000000    1143.000000
mean         3.311015      0.657708     10.442111      5.657043     804.969379
std          0.156664      0.170399      1.082196      0.805824     463.997116
min          2.740000      0.330000      8.400000      3.000000      0.000000
25%          3.205000      0.550000      9.500000      5.000000     411.000000
50%          3.310000      0.620000     10.200000      6.000000     794.000000
75%          3.400000      0.730000     11.100000      6.000000    1209.500000
max          4.010000      2.000000     14.900000      8.000000    1597.000000
```

Code

```
df.isnull().sum()
```

```
fixed acidity      0
volatile acidity   0
citric acid        0
residual sugar     0
chlorides          0
free sulfur dioxide 0
total sulfur dioxide 0
density           0
pH                0
sulphates         0
alcohol           0
quality           0
Id                0
dtype: int64
```

Code

```
df.dropna(inplace=True)
```

Code

```
df.duplicated().sum()

0
```

Code

```
sns.boxplot(x=df["citric acid"])
plt.show()
```

Code

```
sns.boxplot(x=df["pH"])
plt.show()
```

Code

```
sns.boxplot(x=df["quality"])
plt.show()
```

Code

```
sns.barplot(x=df["quality"], y=df["alcohol"])
plt.show()
```

Code

```
sns.barplot(x=df["density"], y=df["pH"])
plt.show()
```

Code

```
sns.boxplot(x=df["density"])
plt.show()
```

Code

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

Code

```
df.duplicated().sum()

0
```

Code

```
sns.countplot(x="density" , hue="quality", data=df)
plt.show()
```

Code

```
import matplotlib.pyplot as plt
plt.figure(figsize=(15,7))
sns.heatmap(df.corr(), annot=True , linewidth=0.5, linecolor="black", fmt='2f')

<AxesSubplot:>
```

Code

```
sns.countplot(x='quality', data=df)
plt.show()
```

Code

```
sns.histplot(df["quality"],kde=True)
plt.show()
```

Code

```
df.hist(figsize=(12,8))
plt.show

<function matplotlib.pyplot.show(close=None, block=None)>
```

Code

```
index=0
labels=df['quality']
features=df.drop('quality' ,axis=1)

for col in features.items():
    plt.figure(figsize=(10,5))
    sns.barplot(x=labels,y=col[index],data=df,color="blue")
    plt.tight_layout()
    plt.show()
```

Code

```
fig, ax=plt.subplots(ncols=6, nrows=3, figsize=(15,10))
index=0
ax=ax.flatten()
for col, value in df.items():
    sns.histplot(value, ax=ax[index])
    index+=1
plt.tight_layout(pad=0.5, w_pad=0.7, h_pad=5.0)
plt.show()
```

Code

```
sns.scatterplot(x="citric acid" ,y="alcohol" , data=df)
plt.show()
```

Code

```
Q1_chlorides = df['sulphates'].quantile(0.25)
Q3_chlorides = df['sulphates'].quantile(0.75)
IQR_chlorides = Q3_chlorides - Q1_chlorides

print(f"IQR for chlorides: {IQR_chlorides}")

IQR for chlorides: 0.17999999999999994
```

Code

```
sns.scatterplot(x="citric acid" ,y="quality" , data=df)
sns.scatterplot(x="residual sugar" ,y="quality" , data=df)
sns.scatterplot(x="chlorides" ,y="quality" , data=df)
sns.scatterplot(x="free sulfur dioxide" ,y="quality" , data=df)
sns.scatterplot(x="total sulfur dioxide" ,y="quality" , data=df)
sns.scatterplot(x="density" ,y="quality" , data=df)
sns.scatterplot(x="pH" ,y="quality" , data=df)
sns.scatterplot(x="sulphates" ,y="quality" , data=df)
sns.scatterplot(x="alcohol" ,y="quality" , data=df)

<AxesSubplot:xlabel='citric acid', ylabel='quality'>
```

Code

```
sns.displot(df["fixed acidity"])

<seaborn.axisgrid.FacetGrid at 0x21a96090d00>
```

Code

```
sns.displot(df["volatile acidity"])
sns.displot(df["citric acid"])
sns.displot(df["residual sugar"])
sns.displot(df["alcohol"])
sns.displot(df["quality"])

<seaborn.axisgrid.FacetGrid at 0x21a9619eca0>
```

Code

```
X=df.drop('quality',axis=1)
y=df['quality']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
results=[]
```

Code

```
def evaluate(name,model):
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    print(name,acc)
    print(classification_report(y_test,pred))
    cm=confusion_matrix(y_test,pred)
    sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
    plt.title(name)
    plt.show()
    results.append([name,acc,model.get_depth(),model.tree_.node_count])
    return model
```

Code

```
tree=evaluate('Original',DecisionTreeClassifier(criterion='entropy',random_state=42))
plt.figure(figsize=(35,15))
plot_tree(tree,feature_names=X.columns,filled=True,max_depth=3)
plt.show()
```

```
Original 0.6200873362445415
      precision    recall  f1-score   support

     3         0.00      0.00      0.00         1
     4         0.20      0.14      0.17         7
     5         0.69      0.80      0.74        97
     6         0.62      0.52      0.56        92
     7         0.48      0.52      0.50        29
     8         0.00      0.00      0.00         3

 accuracy            0.62        229
 macro avg           0.33        229
weighted avg           0.61        229
```

```
C:\Users\cse\anaconda3\lib\site-packages\sklearn\metrics\_classification.py:1248: UndefinedMetricWarning: Precision
_warn_prf(average, modifier, msg_start, len(result))
C:\Users\cse\anaconda3\lib\site-packages\sklearn\metrics\_classification.py:1248: UndefinedMetricWarning: Precision
_warn_prf(average, modifier, msg_start, len(result))
C:\Users\cse\anaconda3\lib\site-packages\sklearn\metrics\_classification.py:1248: UndefinedMetricWarning: Precision
_warn_prf(average, modifier, msg_start, len(result))
```

Code

```
plt.figure(figsize=(25,15))

plot_tree(
    tree,
    feature_names=X.columns,
    class_names=[str(i) for i in sorted(y.unique())],
    filled=True,
    max_depth=3,
    fontsize=12
)

plt.show()
```

Code

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

DecisionTreeClassifier(random_state=42)
```

Code

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

DecisionTreeClassifier(random_state=42)
```

Code

```
import pandas as pd

importance = pd.Series(
    dt_model.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

print(importance)

alcohol            0.160898
total sulfur dioxide 0.128966
volatile acidity   0.093912
pH                0.084272
density           0.080849
sulphates         0.073052
citric acid       0.072273
Id                0.072087
```

```
fixed acidity      0.070660
residual sugar    0.070639
chlorides         0.052999
free sulfur dioxide 0.039393
dtype: float64
```

Code

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

importance.sort_values().plot(kind='barh')

plt.xlabel('Feature Importance')
plt.ylabel('Features')
plt.title('Decision Tree Feature Importance')

plt.show()
```

Code

```
from sklearn.metrics import accuracy_score

train_pred = tree.predict(X_train)
test_pred = tree.predict(X_test)

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

Training Accuracy: 1.0
Testing Accuracy: 0.6200873362445415
```

Code

```
print(tree)

DecisionTreeClassifier(criterion='entropy', random_state=42)
```

Code

```
# HYPERPARAMETER TUNING
```

Code

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'criterion': ['entropy'],
    'max_depth' : [3,5,7,10,None],
    'min_samples_split': [2,5,10,20],
    'min_samples_leaf' : [1,2,4,8],
    'max_features': [None, 'sqrt', 'log2'],
    'ccp_alpha': [0,0.001,0.005,0.01]
}

grid = GridSearchCV(DecisionTreeClassifier(),param_grid,cv=5,n_jobs=-1,scoring='accuracy')
grid.fit(X_train,y_train)
print("Best parameters")
print(grid.best_params_)
print("Best Score")
print(grid.best_score_)
best_tree=grid.best_estimator_

Best parameters
{'ccp_alpha': 0.005, 'criterion': 'entropy', 'max_depth': 5, 'max_features': 'log2', 'min_samples_leaf': 8, 'min_sa
Best Score
0.5820392722032066
```

Code

```
#Cross validation
```

Code

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(best_tree,X,y,cv=10,scoring='accuracy')
print(scores)
print("Average Accuracy=",scores.mean())
print("Std =", scores.std())

[0.51304348 0.49565217 0.54782609 0.57017544 0.53508772 0.68421053
 0.51754386 0.5          0.57894737 0.47368421]
Average Accuracy= 0.5416170861937453
Std = 0.056990178872104044

C:\Users\cse\anaconda3\lib\site-packages\sklearn\model_selection\_split.py:666: UserWarning: The least populated class in y has only 1
warnings.warn(("The least populated class in y has only %d"
```

Code

```
#learning Curve
```

Code

```
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt

train_sizes, train_scores, test_scores = learning_curve(
    best_tree,
    X,
    y,
    cv=5,
    scoring='accuracy'
)

plt.figure(figsize=(8, 5))

plt.plot(
    train_sizes,
    train_scores.mean(axis=1),
    marker='o',
    label='Training Accuracy'
)

plt.plot(
    train_sizes,
    test_scores.mean(axis=1),
    marker='o',
    label='Validation Accuracy'
)

plt.xlabel("Training Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve")
plt.legend()

plt.show()
```

Code

```
#Validation Curve
```

Code

```
from sklearn.model_selection import validation_curve

param_range=[1,2,3,4,5,6,7,8,9,10,None]
train_scores,test_scores=validation_curve(best_tree,X,y,param_name='max_depth',param_range=param_range,cv=5,scoring='accuracy')
plt.figure(figsize=(8,5))
```



```
plt.plot(param_range, train_scores.mean(axis=1), marker='o', label='Training Accuracy')
plt.plot(param_range, test_scores.mean(axis=1), marker='o', label='Validation Accuracy')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Code

```
#normalized confusion matrix
```

Code

```
from sklearn.metrics import confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, test_pred)

cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

plt.figure(figsize=(8, 5))
sns.heatmap(cm_normalized, annot=True, fmt='.2f')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Normalized Confusion Matrix")

plt.show()
```